

Static view: Utilities

Application Layer

Middle-ware Layer

<<Module>>

<<uses>>

oradio_logging.py
<<module>>

- + oradio_log.debug()
- + oradio_log.warning()
- + oradio_log.info()
- + oradio_log.error()

system_sounds.py
<<module>>

- + play_sound()

Every modules/classes can use these utilities

Hardware Abstraction Layer

Static view: Throttle

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<uses>>

throttled_monitor.py
<<module>>

- throttled_monitor: RpiThrottlingMonitor()

<<uses>>

throttled_monitor.RpiThrottlingMonitor
<<singleton>>

- stop_event: Event()

- init()
+ start()
+ stop()
+ run()
+ force_throttled()
+ clear_throttled()

The rpi-throttling-monitor thread
check the throttle state in a loop

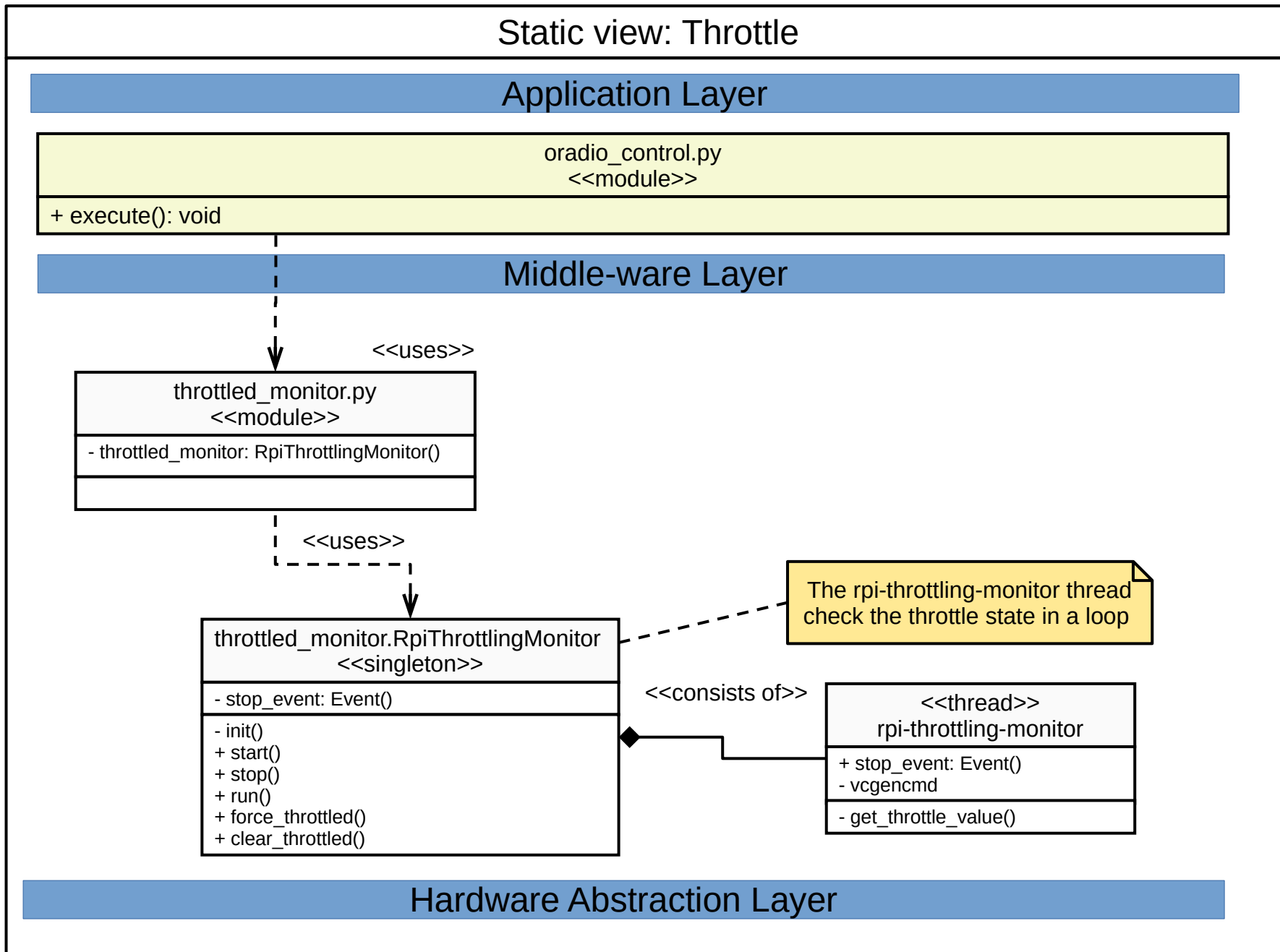
<<consists of>>

<<thread>>

rpi-throttling-monitor

+ stop_event: Event()
- vcgencmd
- get_throttle_value()

Hardware Abstraction Layer



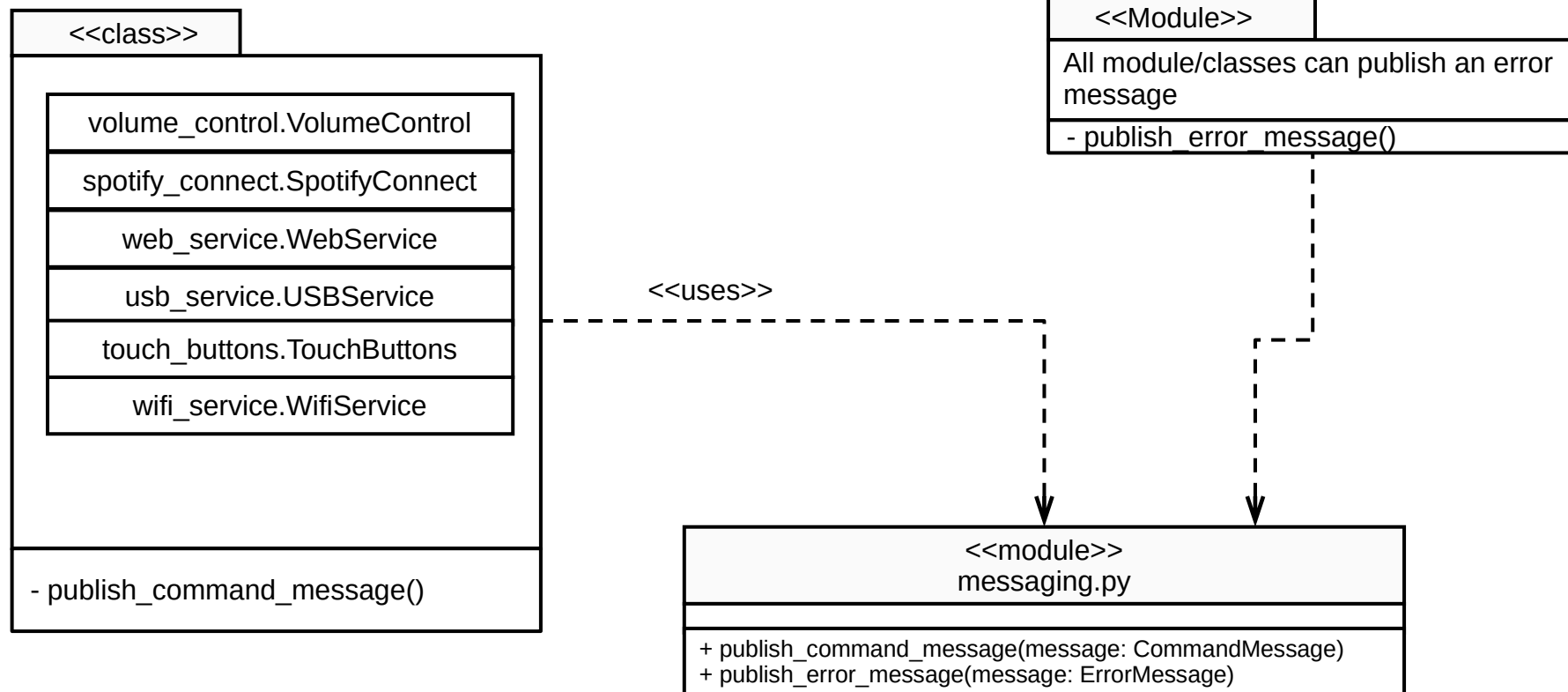
Static view: Messages

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer



Hardware Abstraction Layer

Static view: Remote monitoring

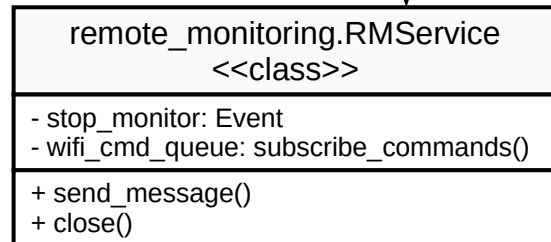
Application Layer

oradio_control.py
<<module>>

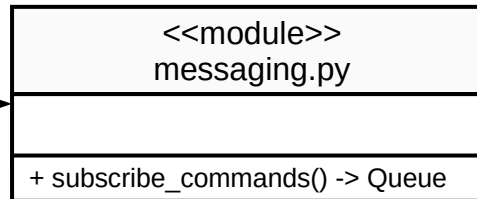
+ execute(): void

Middle-ware Layer

<<uses>>

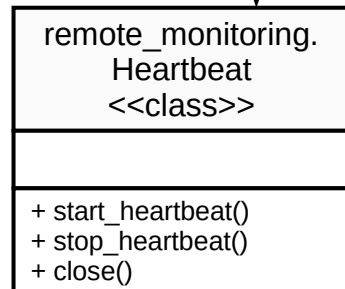


<<uses>>

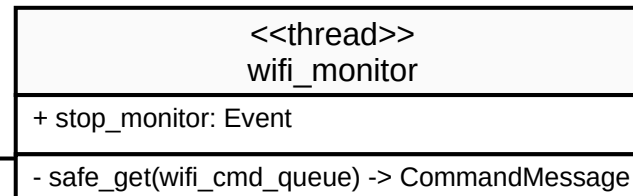


The wifi_monitor thread waits for wifi topic and processes topic

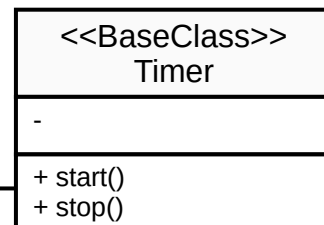
<<uses>>



<<consists of>>



<<consists of>>



Hardware Abstraction Layer

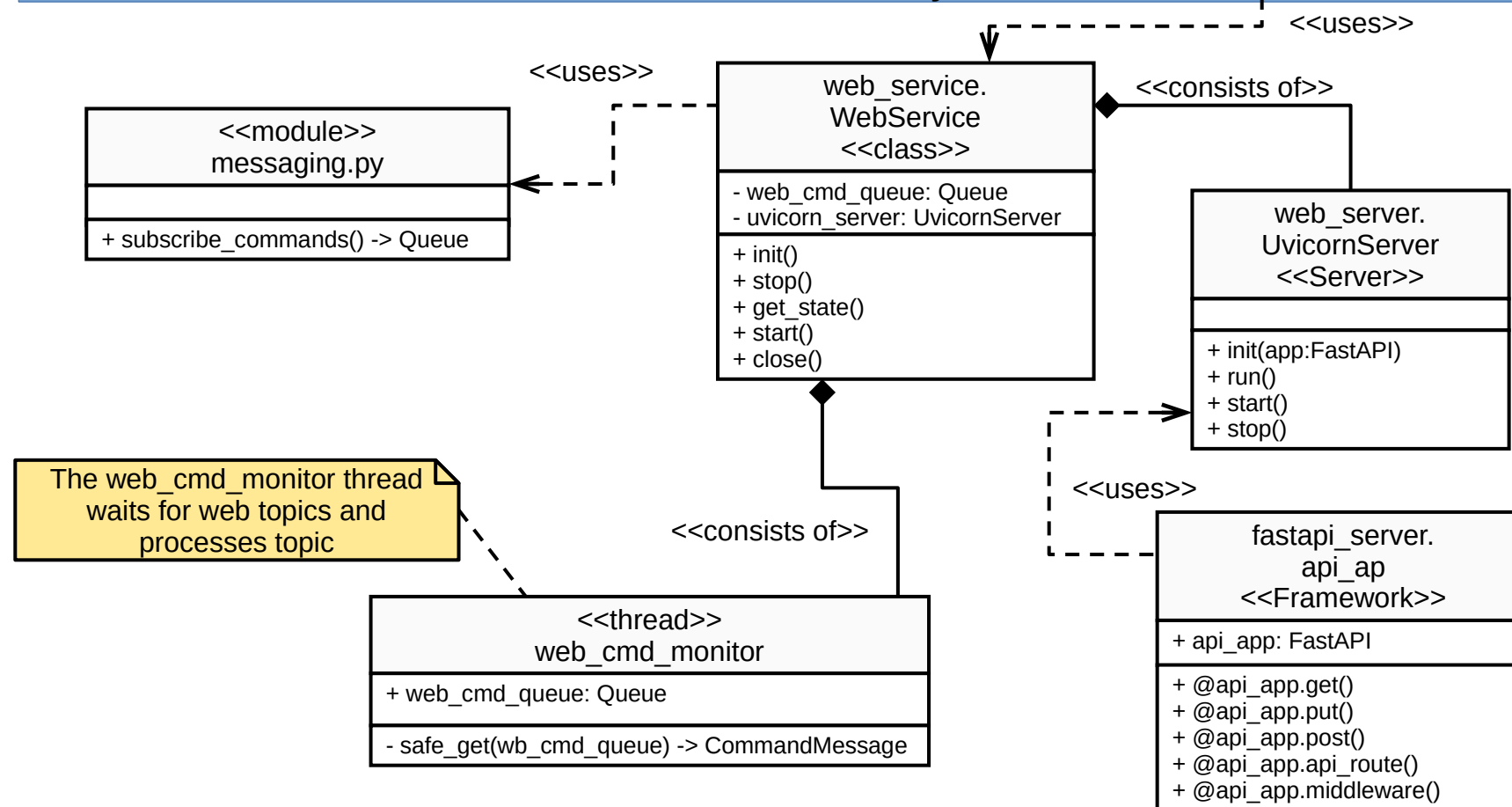
Static view: Web Service

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer



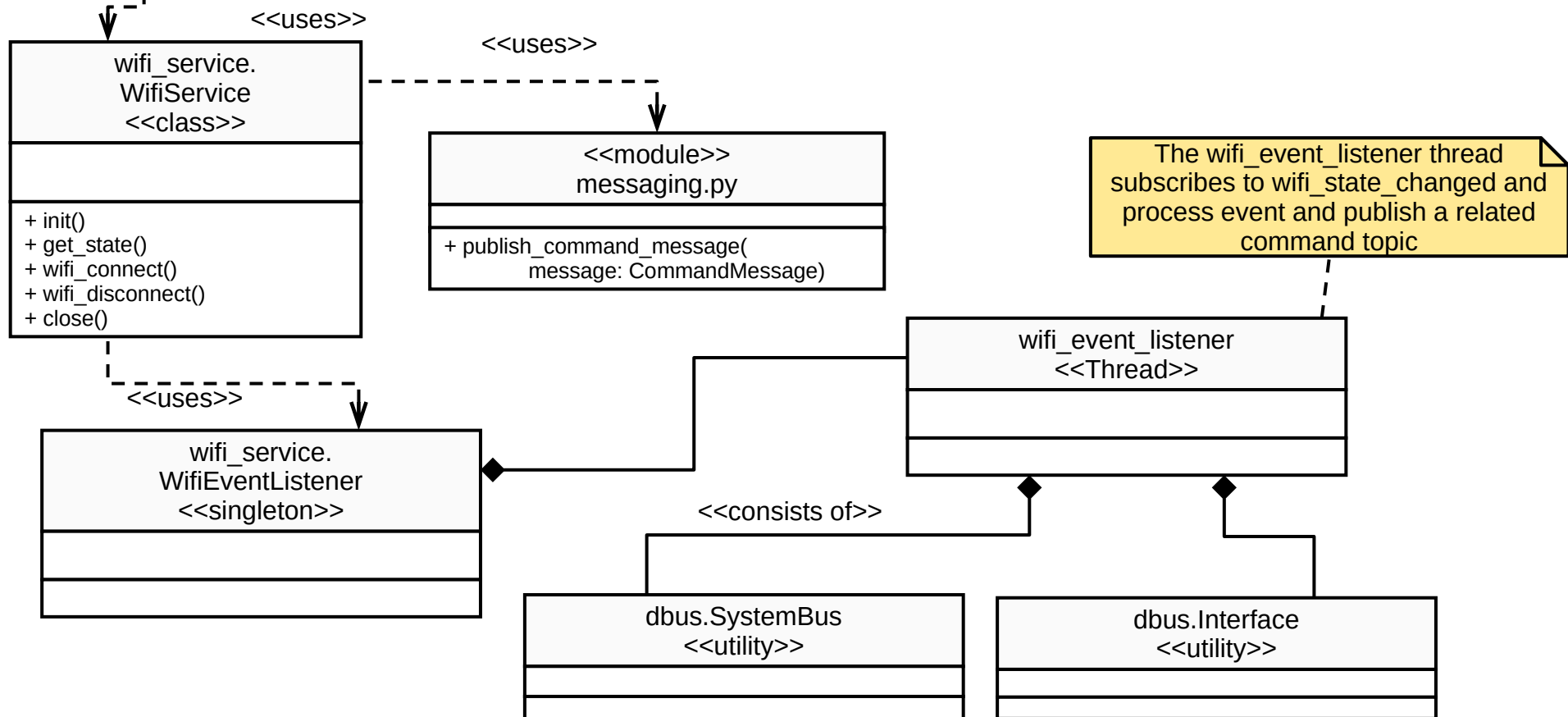
Static view: WiFi Service

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer



Static view: USB

Application Layer

oradio_control.py <<module>>

execute()

- usb_service = USBService()

Middle-ware Layer

<<uses>>

<<consists of>>

usb_service.USBService
<<class>>

- usb_observer: USBObserver()
- usb_monitor: USBMonitor()

+ init()
+ get_state()
+ close()
- usb_inserted()
- usb_removed()
- send_message()

<<consists of>>

<<singleton>>
usb_service.USBMonitor

- on_created()
- usb_deleted()

<<singleton>>
usb_service.USBObserver

- observer: Observer
+ start()
+ stop()

<<module>>
wifi_service.py

+ networkmanager_add(
network:str, password:str)

<<Library>>
watchdog.observers.
Observer

+ schedule()

<<module>>
messaging.py

+ publish_command_message(
message: CommandMessage)

<<is a>>

<<BaseClass>>
PatternMatchingEventHandler

+ handle_event(event)

Hardware Abstraction Layer

Static view: Volume

Application Layer

oradio_control.py <<module>>

execute()

- volume_control = VolumeControl()

Middle-ware Layer

<<uses>>

volume_control.VolumeControl
<<class>>

+ set_notify()
+ start()
+ stop()

<<has>>

i2c_service.I2CService
<<class>>

+ read_byte(device: int, register: int)
+ write_byte(device: int, register: int)
+ read_block(device: int, register: int, length: int)
+ write_block(device: int, register: int, length: int)

<<module>>
messaging.py

+ publish_command_message(
 message: CommandMessage)

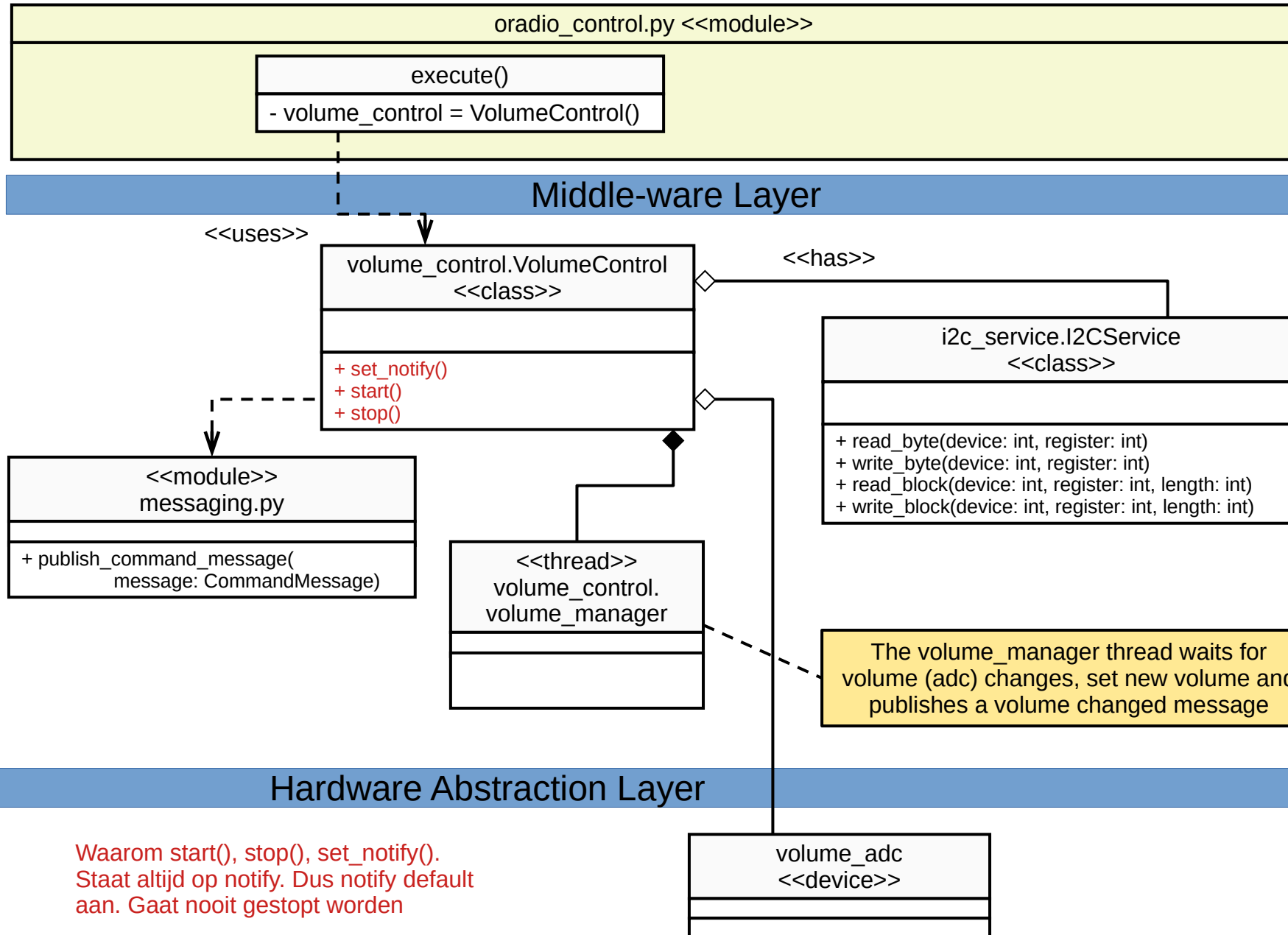
<<thread>>
volume_control.
volume_manager

The volume_manager thread waits for
volume (adc) changes, set new volume and
publishes a volume changed message

Hardware Abstraction Layer

volume_adc
<<device>>

Waarom start(), stop(), set_notify().
Staat altijd op notify. Dus notify default
aan. Gaat nooit gestopt worden



Static view: Error

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<uses>>

error_service.py.ErrorService
<<class>>

- error_queue:subscriber_errors()

- init()
- error_handler()

<<thread>>

error_service.error_handler

<<uses>>

<<module>>
messaging.py

+ ErrorMessage

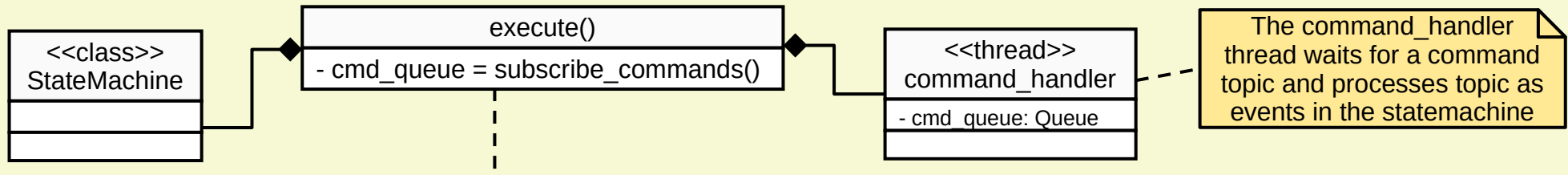
+ safe_get(error_queue: Queue)

Hardware Abstraction Layer

Static view: Messaging

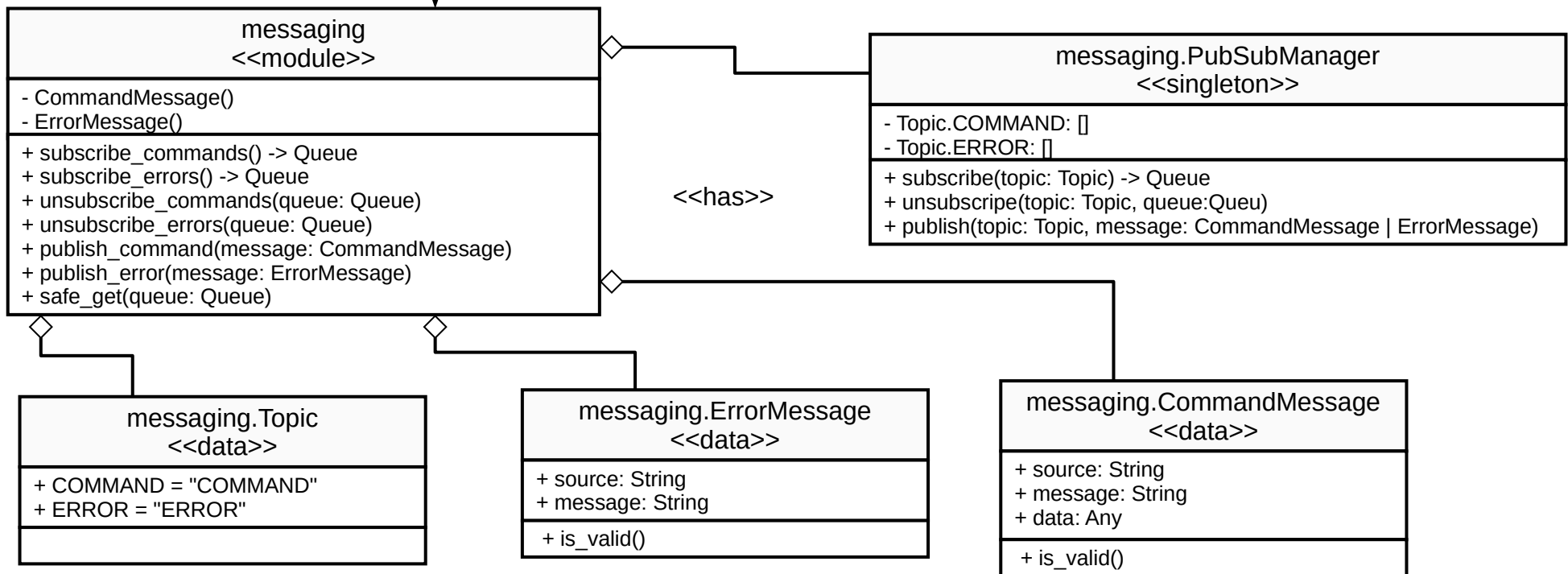
Application Layer

oradio_control.py <<module>>



Middle-ware Layer

<<uses>>



Hardware Abstraction Layer

Static view: Power Supply

Application Layer

oradio_control.py <<module>>

execute()

- power_supply_service = PowerSupplyService()

Middle-ware Layer

↓ <<uses>>

power_supply_control.PowerSupplyService
<<class>>

+ set_nom_voltage()
+ set_max_voltage()
+ set_standby_voltage()
+ read_status()

<<has>>

<<singleton>>

i2c_service.I2CService

+ read_byte(device: int, register: int)
+ write_byte(device: int, register: int)
+ read_block(device: int, register: int, length: int)
+ write_block(device: int, register: int, length: int)

Hardware Abstraction Layer

static view: Front-panel

Application Layer

oradio_control.py <<module>>

execute()

- leds = LEDControl()
- backlighting = Backlighting()
- touch_buttons = TouchButtons()

<<uses>>

Middle-ware Layer

led_control.
LEDControl
<<class>>

- stop_evt: Event()
- + turn_on_led()
- + turn_off_led()
- + turn_off_all_leds()
- + get_led_state()
- + oneshot_on_led()
- + control_blinking_led()

touch_buttons.
TouchButtons
<<class>>

- stop_evt: Event()
- + init()
- button_event_callback()

backlighting.
BackLighting
<<class>>

- thread: backlighting_manager
- running: Event()

<<consists of>>

<<thread>>
backlighting.
backlighting_manager

- + running: Event()

<<uses>>

<<module>>
messaging.py

- + publish_command_message(
message: CommandMessage)

<<has>>

<<singleton>>
i2c_service.I2CService

- + read_byte(device: int, register: int)
- + write_byte(device: int, register: int)
- + read_block(device: int, register: int, length: int)
- + write_block(device: int, register: int, length: int)

<<thread>>
led_control.
blink

- + stop_evt: Event()

<<singleton>>
gpio_service.
GPIOService

Hardware Abstraction Layer

Waarom backlighting thread
met stop en off. Staat eigenlijk
altijd aan! Wordt ook in de
module gestart!

light_sensor
<<device>>

backlight_dac
<<device>>

Static View: Music Players

Application Layer

oradio_control.py <<module>>

execute()

- mpd_monitor = MPDMonitor()
- mpd_control = MPDControl()
- spotify_connect = SpotifyConnect()

Middle-ware Layer

<<uses>>

mpd_control.MPDControl
<<class>>

+ init(class: MPDService)
+ is_web_radio()
+ next()
+ play()
+ stop()
+ pause()

<<is A>>

<<consists of>>

<<thread>>
mpd_control.
remove_song_when_
finished

<<uses>>

mpd_monitor.MPDMonitor
<<class>>

- running: Event
+ init(class: MPDService)
- listen()

<<is A>>

mpd_service.MPDService
<<class>>

- client: MPDClient
+ init()
+ get_stats()
- execute()

<<consists of>>

<<thread>>
mpd_monitor.
listen

+ running: Event

mpd.MPDClient
<<library>>

+ init()
+ idle()
+ update()
+ next()
+ play()
+ stop()
+ pause()
+ status()

<<uses>>

spotify_connect.SpotifyConnect
<<class>>

+ play()
+ pause()
+ get_state()

Hardware Abstraction Layer